Fase 1: El ecosistema DNS (OSINT y Web)

Jerarquía DNS — Organismos y TLDs

¿Qué organismo internacional coordina y asigna los parámetros a nivel global del sistema de nombres de dominio e IPs?

ICANN (Internet Corporation for Assigned Names and Numbers) es efectivamente la organización sin fines de lucro que coordina globalmente el DNS, pero su función va más allá:

- **Coordina el espacio de identificadores únicos de Internet**: Esto incluye no solo los nombres de dominio, sino también las direcciones IP (a través de su supervisión de IANA) y los parámetros de protocolo.
- **Supervisa IANA** (Internet Assigned Numbers Authority): IANA es la función técnica que ICANN gestiona mediante contrato con el Departamento de Comercio de EE.UU. (históricamente). IANA asigna bloques de IPs a los RIRs (Regional Internet Registries como RIPE NCC en Europa) y delega la gestión de TLDs.
- **Acredita Registrars**: ICANN no vende dominios directamente, pero certifica a las empresas (Registrars) que pueden vender dominios .com, .net, etc.
- **Desarrolla políticas**: A través de procesos multistakeholder (GNSO, ccNSO), ICANN facilita que gobiernos, empresas, técnicos y usuarios decidan cómo se gestiona el DNS globalmente.

Gestión de TLDs específicos:

TLD	Tipo	Registry (organismo/gestor)	Notas técnicas
.es	ccTLD (geográfico, España)	**Red.es** (operado por ES-NIC, dependiente de Red.es/Ministerio de Asuntos Económicos)	Gestiona toda la zona .es, mantiene la base de datos central, pero NO vende al público directamente
.cat	sTLD (cultural/lingüístico, comunidad catalana)	**Fundació puntCAT**	Primer TLD basado en comunidad lingüística/cultural (aprobado 2005). Requiere que el contenido esté en catalán o relacionado con la cultura catalana
.edu	sTLD (educativo, EE.UU.)	**Educause** (bajo contrato con el Departamento de Comercio de EE.UU.)	Restringido a instituciones de educación superior acreditadas en EE.UU. (no es global, es específico de Estados Unidos)

ifp.es	Dominio de segundo nivel (NO es TLD)	El **Registry** es Red.es (.es), pero la gestión técnica directa (configurar DNS, registros A, MX, etc.) la controla el **titular del dominio** (IFP) a través de su Registrar	Un dominio de segundo nivel como ifp.es depende del TLD .es para existir, pero el propietario decide qué servidores DNS lo resuelven

Por qué la pregunta es capciosa: Un TLD (Top-Level Domain) es el último segmento del nombre (después del último punto). En `ifp.es`, el TLD es `.es`, no `ifp.es`. El `ifp` es un SLD (Second-Level Domain). La confusión común es pensar que porque "ifp.es" parece completo, es un TLD, pero jerárquicamente está dos niveles por debajo de la raíz.

Herramientas OSINT (Whois y DNS Lookup)

¿Qué información te brinda una consulta Whois sobre un dominio?

Una consulta Whois devuelve metadatos del registro del dominio, pero la información exacta depende de:

- **El TLD** (cada Registry decide qué campos mostrar)
- **Las leyes de privacidad** (GDPR en Europa oculta datos personales desde 2018)

Información típica disponible:

1. **Datos de registro**:
    - **Domain Name**: El nombre exacto del dominio
    - **Registry Domain ID**: Identificador único en la base de datos del Registry
    - **Registrar WHOIS Server**: El servidor Whois del Registrar
    - **Registrar URL**: Web del Registrar
    - **Registrar**: Nombre de la empresa Registradora (ej: GoDaddy, Namecheap, DonDominio)
    - **Registrar IANA ID**: Código numérico que ICANN asigna a cada Registrar acreditado
2. **Fechas del ciclo de vida**:
    - **Creation Date**: Cuándo se registró por primera vez
    - **Registry Expiry Date**: Cuándo caduca (si no se renueva, pasa a estado de redención)
    - **Updated Date**: Última modificación de los datos del registro
3. **Servidores DNS**:
    - **Name Server**: Lista de servidores autoritativos (ej: `ns1.example.com`, `ns2.example.com`)
    - **DNSSEC**: Si está firmado con DNSSEC (unsigned/signed)
4. **Estado del dominio** (Status codes EPP):
    - `clientTransferProhibited`: El dominio no puede transferirse sin autorización del titular

    - `serverDeleteProhibited`: El Registry bloquea la eliminación
    - `ok`: Estado normal, sin restricciones
    - `redemptionPeriod`: El dominio caducó y está en período de gracia (se puede recuperar pagando extra)
5. **Contactos** (parcialmente ocultos por GDPR):
    - **Registrant Organization**: Nombre de la empresa/entidad titular
    - **Registrant State/Province**: Provincia/estado (el país sí suele verse)
    - **Registrant Country**: Código ISO del país (ej: ES, US)
    - **Admin Email / Tech Email**: Correos de contacto administrativo y técnico (a menudo enmascarados con servicios de privacidad como `contact@privacyprotect.org`)

Limitación importante: Desde el GDPR (2018), los datos personales de titulares en la UE (nombre completo, dirección postal, teléfono, email personal) están **ocultos** en Whois público. Solo se ven datos de la organización si es una empresa, y el país. Para obtener datos completos, hay que solicitarlo mediante procesos legales o al propio titular.

Diferencia entre Registry y Registrar (Registrador):

Concepto	Registry	Registrar (Registrador)
Función principal	Mantiene la **base de datos central** de un TLD específico (ej: todos los .es, todos los .com)	Empresa **comercial** que vende dominios al público final
Relación con el público	**NO vende directamente** a usuarios finales	**SÍ vende** al cliente final (tú le pagas a ellos)
Ejemplos	- `.es` → Red.es (ES-NIC) - `.com` → Verisign - `.cat` → Fundació puntCAT	- GoDaddy - Namecheap - DonDominio - Google Domains (ahora Squarespace)
Responsabilidad técnica	- Define las reglas del TLD (precios, requisitos) - Opera los servidores DNS raíz del TLD - Mantiene la zona DNS del TLD	- Interfaz de usuario para comprar/gestionar dominios - Envía las órdenes de registro al Registry vía protocolo EPP - Ofrece servicios adicionales (hosting, email, SSL)
Modelo de negocio	Sin ánimo de lucro o concesión gubernamental (los ccTLDs)	Ánimo de lucro (margen entre precio de Registry y precio al cliente)

Protocolo técnico	Usa **EPP** (Extensible Provisioning Protocol) para comunicarse con Registrars	Usa **EPP** para enviar comandos al Registry (create, renew, transfer, delete)

Flujo simplificado:

1. Tú pagas al **Registrar** (ej: 12€/año por un .es)
2. El **Registrar** envía una orden EPP al **Registry** (Red.es)
3. El **Registry** actualiza su base de datos central y la zona DNS
4. El dominio empieza a resolver en Internet

Analogía: El Registry es como el **Registro de la Propiedad** (base de datos oficial del Estado), y el Registrar es como una **inmobiliaria** (te vende la casa, pero la escritura se inscribe en el Registro).

¿Qué es DNSSEC y qué problema de seguridad intenta resolver?

DNSSEC (Domain Name System Security Extensions) es un conjunto de extensiones al protocolo DNS que añade **firmas criptográficas** a los registros DNS convencionales (A, AAAA, MX, NS, etc.).

Problema que resuelve: Ataques de envenenamiento de caché (DNS cache poisoning) y spoofing

El problema original de DNS:

- DNS fue diseñado en los años 80, cuando Internet era pequeño y de confianza
- Las consultas DNS viajan en **texto plano** (sin cifrar)
- No hay autenticación: cualquier servidor que responda primero es aceptado como válido
- Un atacante en la red (man-in-the-middle) puede:
    1. Interceptar tu consulta DNS ("¿Cuál es la IP de banco.com?")
    2. Responder **antes** que el servidor legítimo con una IP falsa ("banco.com es 192.168.1.100")
    3. Tu ordenador acepta la respuesta y te conecta al servidor del atacante (phishing)

Cómo DNSSEC lo soluciona:

1. **Firmas digitales (RRSIG)**:
    - Cada zona DNS (ej: la zona de `aliexpress.com`) tiene una **clave privada** y una **clave pública**
    - El propietario del dominio firma cada registro DNS con la clave privada
    - La firma se almacena como un registro **RRSIG** (Resource Record Signature)
2. **Cadena de confianza (Chain of Trust)**:
    - La clave pública de `aliexpress.com` está firmada por la clave de `.com`

    - La clave pública de `.com` está firmada por la clave de la raíz (`.`)
    - Los resolvers DNS tienen una **trust anchor** (ancla de confianza): la clave pública de la raíz, que se distribuye de forma segura (viene preinstalada en BIND, Unbound, etc.)
3. **Validación**:
    - Cuando tu resolver DNS (ej: 8.8.8.8) consulta `aliexpress.com`:
        - Recibe el registro A (la IP) + el registro RRSIG (la firma)
        - Recibe el registro DNSKEY (la clave pública de la zona)
        - Verifica criptográficamente que la firma coincide con el registro usando la clave pública
        - Si la firma es válida → la respuesta es auténtica
        - Si la firma no coincide o falta → el resolver **descarta la respuesta** y devuelve un error (SERVFAIL)

Registros nuevos que introduce DNSSEC:

- **RRSIG**: Firma criptográfica de un conjunto de registros
- **DNSKEY**: Clave pública de la zona (hay ZSK para firmar registros y KSK para firmar otras claves)
- **DS** (Delegation Signer): Enlace entre zonas (el padre firma la clave del hijo)
- **NSEC/NSEC3**: Prueba de inexistencia (demuestra que un nombre NO existe, evitando ataques de enumeración)

Limitaciones de DNSSEC:

- ❌ **NO cifra** las consultas DNS (solo las firma) → tu proveedor de Internet aún puede ver qué dominios consultas
- ❌ **NO protege disponibilidad** (si el servidor DNS está caído, DNSSEC no ayuda)
- ✅ **Protege integridad y autenticidad**: garantiza que la IP que recibes es la que el propietario del dominio publicó

Para cifrar consultas DNS, se usan protocolos como **DoH** (DNS over HTTPS) o **DoT** (DNS over TLS), que son complementarios a DNSSEC.

Rendimiento DNS — Benchmark de GRC

Proceso del test:

1. Descargas **DNS Benchmark** desde grc.com/dns/benchmark.htm
2. La herramienta consulta **múltiples servidores DNS públicos** (Google DNS, Cloudflare, OpenDNS, Quad9, etc.)
3. Mide:
    - **Latencia de resolución** (tiempo en ms desde que envías la consulta hasta que recibes la respuesta)
    - **Fiabilidad** (tasa de respuestas correctas)
    - **Capacidad de DNSSEC** (si valida firmas)

Interpretación de resultados:

- Los servidores con **menor latencia** están geográficamente más cerca de ti o tienen mejor infraestructura en tu región
- En España, los más rápidos suelen ser:
    - **Cloudflare** (1.1.1.1, 1.0.0.1) → Infraestructura global con PoPs en Madrid y Barcelona
    - **Google DNS** (8.8.8.8, 8.8.4.4) → También con PoPs en España
    - **Quad9** (9.9.9.9) → Enfocado en seguridad, bloquea dominios maliciosos
    - **Vercara/UltraDNS** (156.154.70.x) → Infraestructura empresarial de DDoS protection

Ejemplo de resultados típicos en Madrid:

Servidor	IP	Latencia media	Empresa
Cloudflare Primary	1.1.1.1	~8-12 ms	Cloudflare Inc. (CDN y seguridad)
Cloudflare Secondary	1.0.0.1	~9-13 ms	Cloudflare Inc.
Google DNS Primary	8.8.8.8	~10-15 ms	Google LLC
Quad9	9.9.9.9	~12-18 ms	Quad9 Foundation (sin ánimo de lucro, enfocado en privacidad)

Por qué importa:

- Un DNS más rápido reduce el tiempo de **primera consulta** (antes de que el navegador empiece a cargar la web)
- En navegación diaria, esto se nota en la "sensación de velocidad" al hacer clic en enlaces
- Para servidores, usar DNS rápidos reduce la latencia de aplicaciones que dependen de resolución DNS constante

---

Fase 2: Configuración y Caché

Cambio de servidores DNS

Ver DNS actuales en CLI:

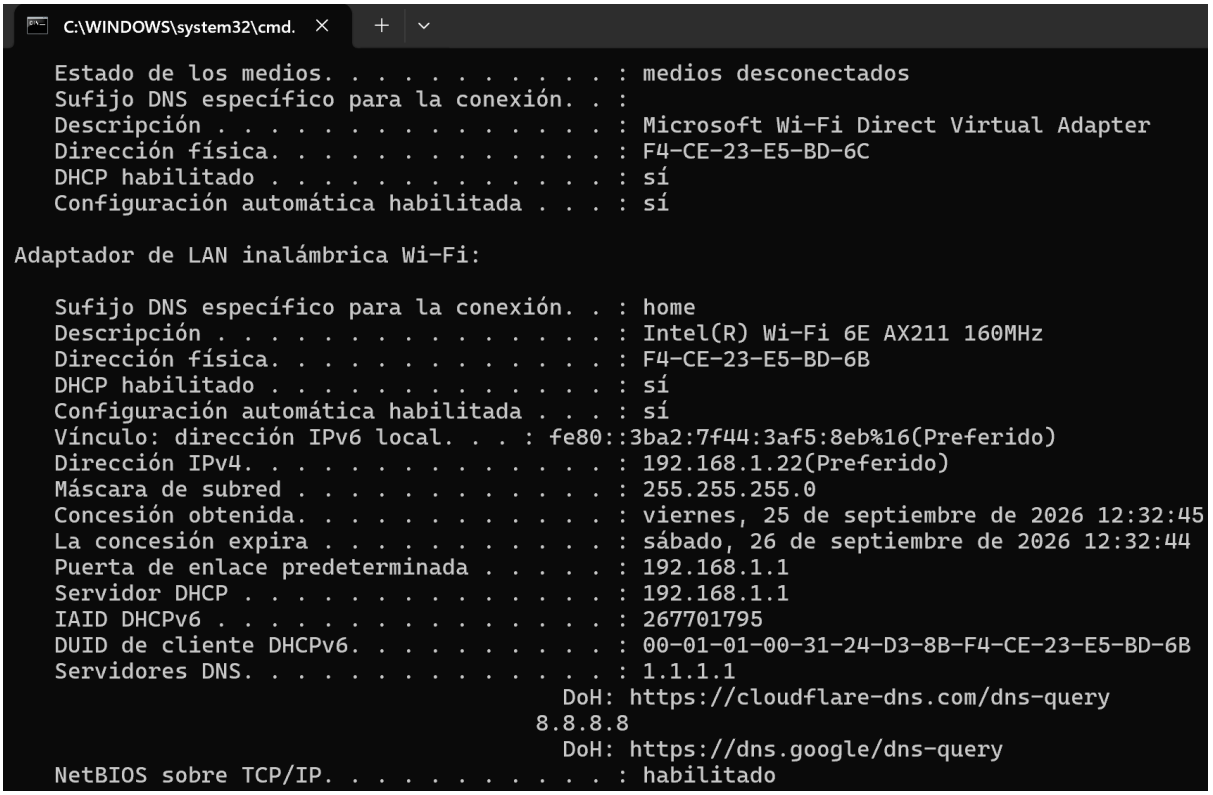
Windows:

bash
```
ipconfig /all
```

Busca la sección de tu adaptador de red (ej: "Adaptador de Ethernet Ethernet0" o "Adaptador de LAN inalámbrica Wi-Fi"). Verás:

text
```
Servidores DNS . . . . . . . . . . . . : 192.168.1.1
                                         8.8.8.8
```

El primero es el **DNS primario**, el segundo el **secundario**.



Linux (depende del gestor de red):

- **systemd-resolved** (Ubuntu 18.04+, Fedora, Debian moderno):
  bash
- `resolvectl status`
- Muestra:
  text

```
Global
       DNS Servers: 1.1.1.1
```

- 8.8.8.8

- **NetworkManager** (la mayoría de distros desktop):
  bash
- `nmcli dev show | grep DNS`
- **Fichero clásico** (solo lectura, no se usa para configurar en sistemas modernos):
  bash
- `cat /etc/resolv.conf`
- Verás `nameserver 192.168.1.1`, pero **no lo edites directamente** en sistemas con systemd o NetworkManager, ya que se sobrescribe al reiniciar.

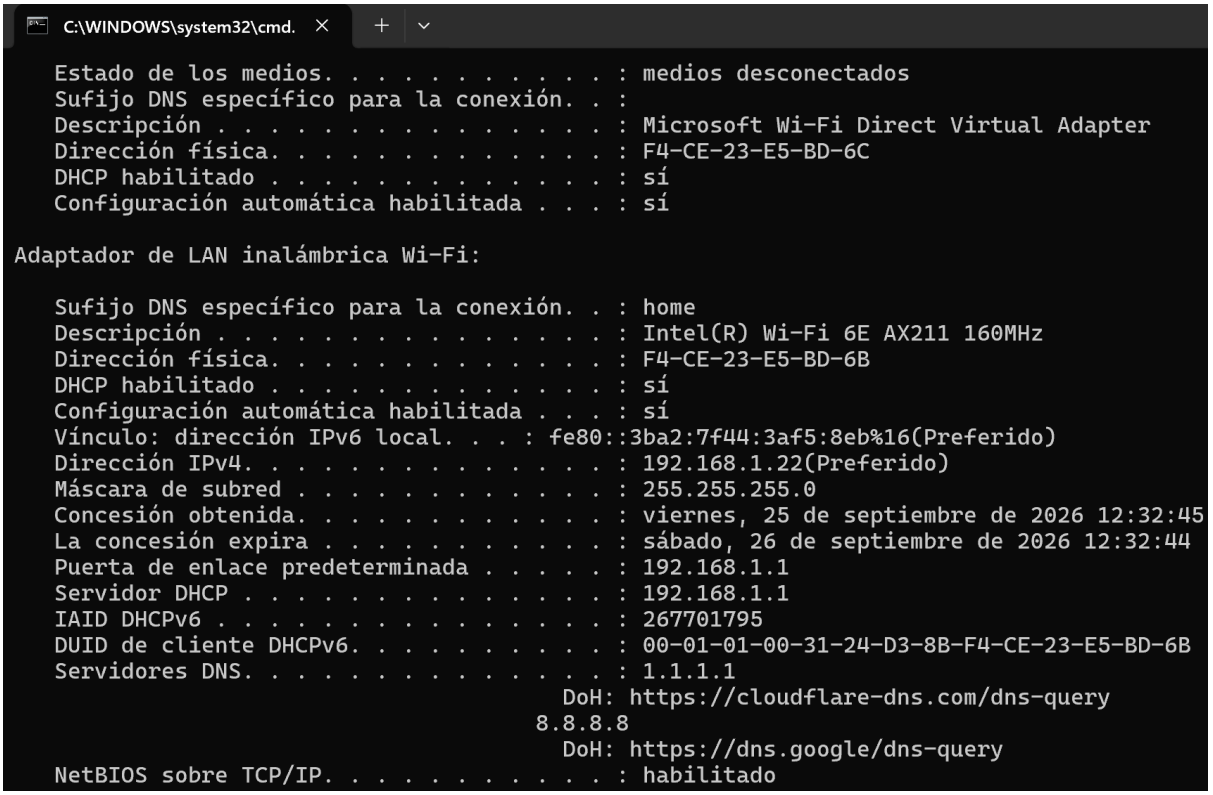
Cambiar DNS en Windows (paso a paso técnico):

1. **Panel de Control → Redes e Internet → Centro de redes y recursos compartidos**
2. Haz clic en tu conexión activa (ej: "Wi-Fi (nombre_de_red)")
3. **Propiedades** → Busca **Protocolo de Internet versión 4 (TCP/IPv4)**
4. **Propiedades** → Marca **"Usar las siguientes direcciones de servidor DNS"**
5. Introduce:
   - **DNS preferido**: `1.1.1.1` (Cloudflare, por ejemplo)
   - **DNS alternativo**: `8.8.8.8` (Google)
6. **Aceptar → Cerrar**

Verificación:

bash
```
ipconfig /flushdns
ipconfig /all
```

Deberías ver los nuevos DNS.



En Android (DNS por Wi-Fi):

1. **Ajustes** → **Redes e Internet** → **Wi-Fi**
2. Mantén pulsada la red Wi-Fi conectada → **Modificar red** (o toca el engranaje)
3. **Opciones avanzadas** → Cambia **Configuración de IP** de **DHCP** a **Estático**
4. Introduce:
    - **IP**: La que tengas actualmente (ej: 192.168.1.50)
    - **Puerta de enlace**: Tu router (ej: 192.168.1.1)
    - **Longitud de prefijo de red**: 24 (equivalente a máscara 255.255.255.0)
    - **DNS 1**: `1.1.1.1`
    - **DNS 2**: `8.8.8.8`
5. **Guardar**

Nota: En Android 9+, también puedes usar **DNS Privado** (DoT) en **Ajustes** → **Redes e Internet** → **DNS Privado**, poniendo `dns.google` o `1dot1dot1dot1.cloudflare-dns.com`. Esto cifra las consultas DNS, no solo cambia el servidor.

Gestión de la caché DNS

Ver caché en Windows:

bash
```
ipconfig /displaydns
```

Muestra una tabla con:

- **Nombre de registro**: El dominio consultado (ej: `www.google.com`)
- **Tipo de registro**: A, AAAA, CNAME, etc.
- **Tiempo de vida (TTL)**: Segundos restantes antes de que caduque de la caché
- **Longitud de datos**: Tamaño del registro
- **Sección de datos**: La IP o valor almacenado

Ejemplo de salida:

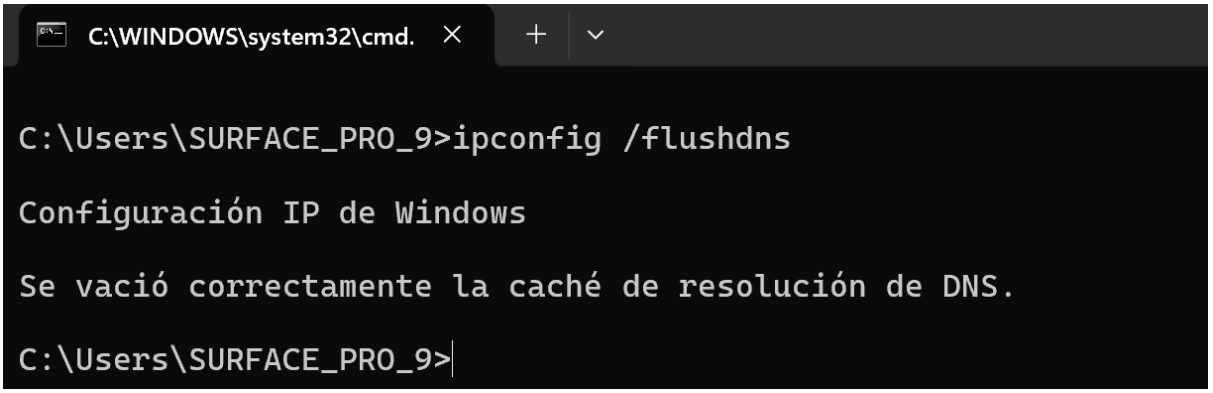
text

```
www.google.com
---------------------------------------
Nombre de registro. . . . . : www.google.com
Tipo de registro. . . . . . : 1
Tiempo de vida. . . . . . . : 299
Longitud de datos . . . . . : 4
Sección. . . . . . . . . . . : Respuestas
IP de recursos. . . . . . . : 142.250.181.68
```



Vaciar caché:

- **Windows**: `ipconfig /flushdns`
- **Linux (systemd-resolved)**: `resolvectl flush-caches`
- **Linux (nscd)**: `sudo systemctl restart nscd`



¿Por qué es útil vaciar la caché en administración de sistemas?

1. **Forzar propagación de cambios**:
   - Si cambias un registro DNS (ej: mueves tu web de 1.2.3.4 a 5.6.7.8), los clientes con caché seguirán usando la IP antigua hasta que el TTL expire
   - En testing, quieres ver el cambio **inmediatamente** → flushdns garantiza que la próxima consulta vaya al servidor DNS y traiga el nuevo valor
2. **Solucionar registros corruptos o erróneos**:
   - A veces un servidor DNS responde mal (ej: devuelve una IP de error o un registro caducado)
   - Tu sistema lo guarda en caché → aunque el servidor se arregle, tu equipo sigue usando el dato malo
   - Flushdns limpia el error y fuerza una nueva consulta limpia
3. **Validar configuración de nuevos DNS**:
   - Si cambias de 8.8.8.8 a 1.1.1.1, pero tu caché tiene registros de hace 10 minutos, no estás probando realmente 1.1.1.1
   - Flushdns asegura que **todas** las consultas nuevas pasen por los servidores recién configurados
4. **Troubleshooting de resolución**:
   - "¿Por qué esta web no carga?" → Puede que la caché tenga un registro A antiguo o un CNAME roto
   - Flushdns + nueva consulta aísla si el problema es de caché local o del servidor DNS

TTL y caché:

- El **TTL** (Time To Live) lo define el propietario del dominio en el registro DNS
- Ejemplo: Si el TTL de `www.ejemplo.com` es 300 segundos (5 minutos), tu sistema guardará esa IP en caché durante 5 minutos
- Pasados 5 minutos, la próxima consulta irá de nuevo al servidor DNS
- TTLs cortos (60-300s): útiles para cambios frecuentes (CDNs, balanceadores)
- TTLs largos (86400s = 24h): para registros estables (reduce carga en servidores DNS)

Fase 3: Troubleshooting con DIG y CLI

Consultas específicas de registros con `dig`

A (registro de dirección IPv4):

bash
```
dig aliexpress.com
```

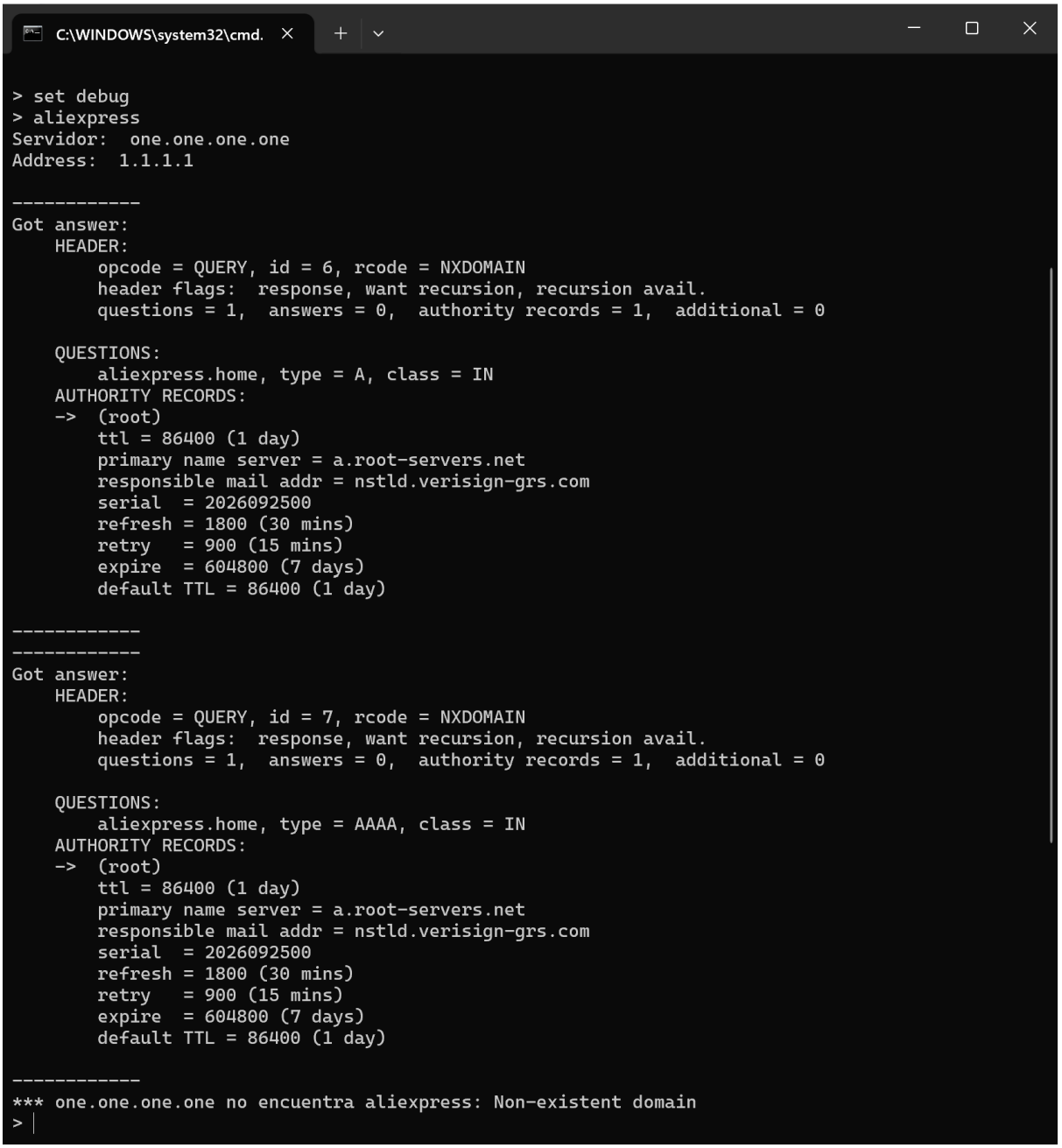
Secciones clave de la salida:

text
```
; <<>> DiG 9.18.12 <<>> aliexpress.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 12345
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; QUESTION SECTION:
;aliexpress.com.                IN      A

;; ANSWER SECTION:
aliexpress.com.           300     IN      A       198.41.30.52
aliexpress.com.           300     IN      A       198.41.31.52

;; Query time: 25 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Tue Sep 22 16:28:00 CEST 2026
;; MSG SIZE  rcvd: 56
```



Explicación:

- **QUESTION SECTION**: Lo que preguntaste (dominio + tipo de registro A)
- **ANSWER SECTION**: Las respuestas. Aquí hay **2 registros A** (el dominio tiene múltiples IPs para balanceo de carga)
    - **300**: TTL en segundos (5 minutos)
    - **IN**: Clase Internet
    - **A**: Tipo de registro
    - **198.41.30.52**: La IP real
- **HEADER**:
    - `qr`: Query Response (es una respuesta, no una consulta)
    - `rd`: Recursion Desired (pediste recursión)

- `ra`: Recursion Available (el servidor soporta recursión)
- `status: NOERROR`: La consulta tuvo éxito

Short (formato breve):

bash
```
dig +short aliexpress.com
```

Salida:

text
```
198.41.30.52
198.41.31.52
```

¿Por qué es útil en scripts de Bash?

- **Parsing sencillo**: Sin cabeceras, sin secciones, solo los datos → fácil de guardar en variables
- **Ejemplo de script**:
  bash

```
#!/bin/bash
IP=$(dig +short aliexpress.com | head -n1)
echo "La primera IP de aliexpress.com es: $IP"

# Comprobar si un dominio resuelve antes de hacer algo
if [ -z "$(dig +short midominio.com)" ]; then
    echo "ERROR: midominio.com no resuelve"
    exit 1
```

- `fi`
- **Automatización**: En monitoreo, despliegues, validaciones previas a conexiones, etc.

MX (Mail Exchange, servidores de correo):

bash
```
dig MX aliexpress.com
```

Salida típica:

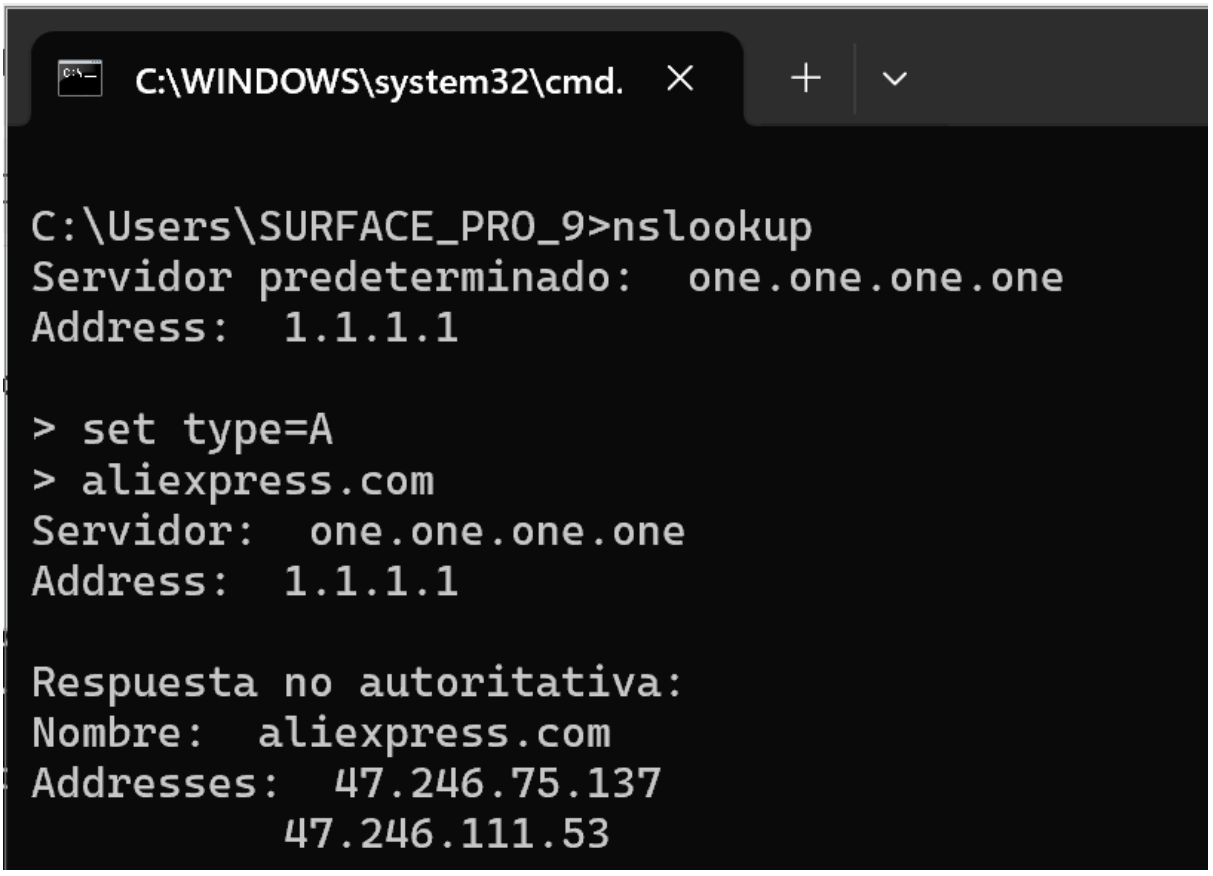
text
```
;; ANSWER SECTION:
```

```
aliexpress.com.          3600    IN      MX      10
mx1.aliexpress.com.
aliexpress.com.          3600    IN      MX      20
mx2.aliexpress.com.
aliexpress.com.          3600    IN      MX      30
```

mx3.aliexpress.com.

```
> set type=MX
> aliexpress.com
Servidor:  one.one.one.one
Address:  1.1.1.1

Respuesta no autoritativa:
aliexpress.com  MX preference = 10, mail exchanger = mx2.mail.aliyun.com
>
```



```
> set type=CNAME
> aliexpress.com
Servidor:  one.one.one.one
Address:  1.1.1.1

aliexpress.com
        primary name server = ns1.alibabadns.com
        responsible mail addr = hostmaster.alibabadns.com
        serial  = 2018051019
        refresh = 3600 (1 hour)
        retry   = 1200 (20 mins)
        expire  = 86400 (1 day)
        default TTL = 360 (6 mins)
> 
```

Campo de Prioridad/Preference:

- El número antes del nombre del servidor (10, 20, 30) es la **prioridad**
- **Menor número = mayor prioridad**
- El servidor de correo que envía un email a `@aliexpress.com` intentará primero **mx1** (prioridad 10)
- Si mx1 está caído, prueba mx2 (20), luego mx3 (30)
- Esto permite **redundancia y balanceo de carga** en el correo entrante

---

NS (Name Servers, servidores autoritativos):

bash
```
dig NS aliexpress.com
```

Salida:

text
```
;; ANSWER SECTION:
aliexpress.com.          86400   IN      NS
ns1.aliexpress.com.
aliexpress.com.          86400   IN      NS
ns2.aliexpress.com.
aliexpress.com.          86400   IN      NS
ns3.aliexpress.com.
```

```
> set type=NS
> aliexpress.com
Servidor:  one.one.one.one
Address:  1.1.1.1

Respuesta no autoritativa:
aliexpress.com  nameserver = ns2.alibabadns.com
aliexpress.com  nameserver = ns1.alibabadns.com
>
```

¿Qué son estos servidores?

- Son los servidores que tienen la **autoridad** sobre la zona `aliexpress.com`
- Ellos son los que **definen** los registros A, MX, CNAME, etc. del dominio
- Cualquier cambio en el DNS de aliexpress.com se hace en estos servidores
- El TTL alto (86400s = 24h) indica que estos registros cambian poco

---

Autoridad y Caché (TTL)

Diferencia entre SOA y NS:

Registro	Función	Dónde aparece	Información clave
SOA (Start of Authority)	Define los **parámetros globales** de la zona DNS	**Solo uno** por zona, al principio de la zona	- MNAME: Servidor DNS primario de la zona - RNAME: Email del responsable (ej: admin.aliexpress.com) - Serial: Número de versión de la zona (incrementa con cada cambio) - Refresh: Cada cuántos segundos los servidores secundarios consultan al primario - Retry: Tiempo de reintento si falla - Expire: Cuándo un secundario deja de servir la zona si no contacta al primario - Minimum TTL: TTL por defecto para registros sin TTL explícito

NS (Name Server)	Lista los **servidores autoritativos** de la zona	**Múltiples** por zona (redundancia)	- Nombres de los servidores que responden con autoridad para esta zona - Se usan en la delegación desde el padre (ej: .com delega a los NS de aliexpress.com)

Ejemplo de SOA:

bash
```
dig SOA aliexpress.com
```
text
```
;; ANSWER SECTION:
aliexpress.com.           3600    IN      SOA
ns1.aliexpress.com. admin.aliexpress.com. (
                                2024092201  ; Serial
                                7200        ; Refresh (2h)
                                3600        ; Retry (1h)
                                1209600     ; Expire (2 semanas)
                                3600 )      ; Minimum TTL (1h)
```

```
> set type=SOA
> aliexpress.com
Servidor:  one.one.one.one
Address:  1.1.1.1

Respuesta no autoritativa:
aliexpress.com
        primary name server = ns1.alibabadns.com
        responsible mail addr = hostmaster.alibabadns.com
        serial  = 2018051019
        refresh = 3600 (1 hour)
        retry   = 1200 (20 mins)
        expire  = 86400 (1 day)
        default TTL = 360 (6 mins)
>
```

TTL y comportamiento de caché:

Consulta 1:

bash
```
dig aliexpress.com
```
text

```
;; ANSWER SECTION:
aliexpress.com.          300     IN      A       198.41.30.52
```

Espera 5 segundos y Consulta 2:

bash
```
dig aliexpress.com
```

text
```
;; ANSWER SECTION:
aliexpress.com.          295     IN      A       198.41.30.52
```

¿Qué pasó?

- El TTL bajó de **300 a 295** segundos
- Esto demuestra que la **segunda respuesta vino de la caché** de tu resolver DNS (no del servidor autoritativo)
- El resolver **decrementa el TTL** cada segundo para saber cuándo debe purgar el registro
- Si el TTL llega a 0, la próxima consulta irá al servidor autoritativo de nuevo

¿Cómo saber si viene de caché o es autoritativa?

- Mira los **flags** en el HEADER:
  - `aa` (Authoritative Answer): **1** = respuesta autoritativa (viene del servidor que gestiona la zona)
  - `aa` ausente o **0** = respuesta recursiva/caché (viene de un resolver intermedio)

Ejemplo autoritativo:

text
```
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 2,
ADDITIONAL: 3
```

Ejemplo caché:

text
```
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
```

Trazabilidad Completa (Trace)

Comando:

bash
```
dig +trace aliexpress.com
```

Salida explicada paso a paso:

text

```
; <<>> DiG 9.18.12 <<>> +trace aliexpress.com
;; global options: +cmd

; Paso 1: Servidores Raíz (.)
.                       518400  IN      NS      a.root-servers.net.
.                       518400  IN      NS      b.root-servers.net.
... (13 servidores raíz, a-m)

;; Received 239 bytes from 192.58.128.30#53(a.root-servers.net) in
25 ms

; Paso 2: Servidores del TLD (.com)
com.                    172800  IN      NS      a.gtld-servers.net.
com.                    172800  IN      NS      b.gtld-servers.net.
... (13 servidores .com, a-m)

;; Received 489 bytes from 198.41.0.4#53(a.root-servers.net) in 30
ms

; Paso 3: Servidores autoritativos de aliexpress.com
aliexpress.com.         172800  IN      NS      ns1.aliexpress.com.
aliexpress.com.         172800  IN      NS      ns2.aliexpress.com.

;; Received 56 bytes from 192.52.178.30#53(a.gtld-servers.net) in 35
ms

; Paso 4: Respuesta final con el registro A
aliexpress.com.         300     IN      A       198.41.30.52

;; Received 56 bytes from 198.41.30.53#53(ns1.aliexpress.com) in 40
ms
```

Explicación del flujo:

1. **Tu ordenador** (resolver recursivo) no sabe dónde está `aliexpress.com` → empieza por la **raíz** (`.`)
2. **Pregunta a un servidor raíz** (hay 13 grupos, a-m.root-servers.net, distribuidos globalmente):
    - "¿Dónde está aliexpress.com?"

    - El raíz responde: "No lo sé, pero los servidores del TLD `.com` sí pueden ayudarte" → devuelve la lista de servidores `.com` (gtld-servers.net)
3. **Pregunta a un servidor del TLD `.com`**:
    - "¿Dónde está aliexpress.com?"
    - El servidor `.com` responde: "No tengo la IP, pero los servidores autoritativos de `aliexpress.com` sí" → devuelve los NS de aliexpress.com (ns1.aliexpress.com, ns2.aliexpress.com)
4. **Pregunta a un servidor autoritativo de `aliexpress.com`**:
    - "¿Cuál es la IP de aliexpress.com?"
    - El servidor autoritativo responde: "Es 198.41.30.52" → **respuesta autoritativa** (tiene el flag `aa`)
5. **Tu resolver guarda la respuesta en caché** (TTL 300s) y te la devuelve

¿Por qué `+trace` es útil?

- Ves **toda la cadena de delegación** (raíz → TLD → autoritativo)
- Identificas dónde está un problema (ej: si el TLD no delega bien, o si el autoritativo no responde)
- Entiendes la **jerarquía real** del DNS (no es magia, es una cadena de consultas)

Fase 4: Análisis de Tráfico de Red (Wireshark)

Captura y filtrado DNS

Filtro de captura:

- En la barra de filtro de Wireshark, escribe: `dns` o `udp.port == 53`
- Esto muestra **solo paquetes DNS** (ignora HTTP, TLS, etc.)

Forzar consulta:

bash
```
nslookup -type=mx google.com
```

Preguntas de Wireshark

1. Capa de Transporte: ¿TCP o UDP? ¿Por qué?

- **Protocolo**: **UDP** (User Datagram Protocol)
- **Por qué UDP**:
    - **Bajo overhead**: UDP no tiene handshake (SYN, SYN-ACK, ACK) como TCP → más rápido para consultas pequeñas

    - **Consultas típicas < 512 bytes**: Un registro A o MX cabe en un solo paquete UDP
    - **Sin necesidad de fiabilidad TCP**: Si se pierde un paquete DNS, el cliente reintenta (no necesita el orden garantizado de TCP)
    - **Excepción**: Si la respuesta > 512 bytes (ej: muchos registros MX, DNSSEC con firmas grandes), se usa **TCP** o **EDNS0** (Extension Mechanisms for DNS) para permitir UDP más grande

En Wireshark:

- Verás `User Datagram Protocol` en el panel de detalles
- Protocolo: **17** (UDP) en el campo IP

---

2. Puertos: origen y destino

- **Puerto de origen (cliente)**: **Aleatorio/efímero** (ej: 54321, 49152-65535)
    - Tu sistema elige un puerto alto no usado para esta consulta
    - Permite múltiples consultas simultáneas sin colisión
- **Puerto de destino (servidor)**: **53** (puerto estándar DNS)
    - Todos los servidores DNS escuchan en UDP 53 (y TCP 53 para respuestas grandes)

En Wireshark:

text
```
User Datagram Protocol, Src Port: 54321, Dst Port: 53
```

---

3. Identificador de transacción (Transaction ID)

- **Qué es**: Un número de **16 bits** (0-65535) que el cliente genera aleatoriamente para cada consulta
- **Función**: Vincular la **respuesta** con la **petición** correcta
    - Tu ordenador puede tener 10 consultas DNS en curso a la vez (diferentes dominios)
    - Cada una tiene un Transaction ID único
    - Cuando el servidor responde, incluye el mismo ID → tu sistema sabe cuál consulta es esta respuesta

En Wireshark:

- Expande **Domain Name System** → **Transaction ID: 0x1234** (ejemplo)
- La respuesta tendrá **el mismo Transaction ID**

---

4. Flags: Authoritative Answer (AA)

- **En la respuesta**, expande **Flags**:
    - **Authoritative Answer (AA)**: **0 o 1**

Significado:

- **AA = 1**: La respuesta es **autoritativa** (viene directamente de un servidor NS que gestiona la zona del dominio consultado)
- **AA = 0**: La respuesta es **recursiva/no autoritativa** (viene de un resolver intermedio que la tiene en caché)

En tu consulta `nslookup -type=mx google.com`:

- Si consultas a tu DNS local (ej: 192.168.1.1 o 8.8.8.8), verás **AA = 0** (es un resolver recursivo, no autoritativo de google.com)
- Si consultas directamente a un NS de google.com (ej: `dig @ns1.google.com MX google.com`), verás **AA = 1**

---

5. Respuestas (Answers): Servidor de correo con prioridad más alta

En Wireshark:

- Expande **Domain Name System → Answers**
- Verás múltiples registros MX, cada uno con:
    - **Mail Exchange**: El nombre del servidor (ej: `alt1.gmail-smtp-in.l.google.com`)
    - **Preference**: El número de prioridad (ej: 10, 20, 30)

Ejemplo real de google.com:

text
```
alt1.gmail-smtp-in.l.google.com, preference = 10
alt2.gmail-smtp-in.l.google.com, preference = 20
alt3.gmail-smtp-in.l.google.com, preference = 30
alt4.gmail-smtp-in.l.google.com, preference = 40
gmail-smtp-in.l.google.com, preference = 5
```

Prioridad más alta (número más bajo): `gmail-smtp-in.l.google.com` con **preference = 5**

Significado:

- Los servidores de correo que envían emails a `@google.com` intentarán primero `gmail-smtp-in.l.google.com`
- Si está caído, probarán alt1 (10), luego alt2 (20), etc.
- Esto es **redundancia**: si un servidor de correo falla, el email no se pierde, va al siguiente

---

Resumen técnico para aprendizaje

Concepto	Qué es	Por qué importa
Concepto	Qué es	Por qué importa
ICANN/IANA	Coordina globalmente DNS e IPs	Sin esto, habría caos de nombres duplicados y IPs solapadas
Registry vs Registrar	Registry = base de datos del TLD; Registrar = tienda que te vende el dominio	Entiendes quién hace qué en la cadena de venta/gestión
DNSSEC	Firmas criptográficas en DNS	Previene que te redirijan a webs falsas (phishing) alterando DNS
TTL	Tiempo de vida de un registro en caché	Afecta a velocidad de propagación de cambios y carga de servidores
dig +trace	Muestra toda la cadena raíz → TLD → autoritativo	Entiendes la jerarquía real y troubleshooteas problemas de delegación
Wireshark DNS	Ver tráfico UDP 53, Transaction ID, AA flag	Depuras problemas de red, validas que DNS funciona como esperas